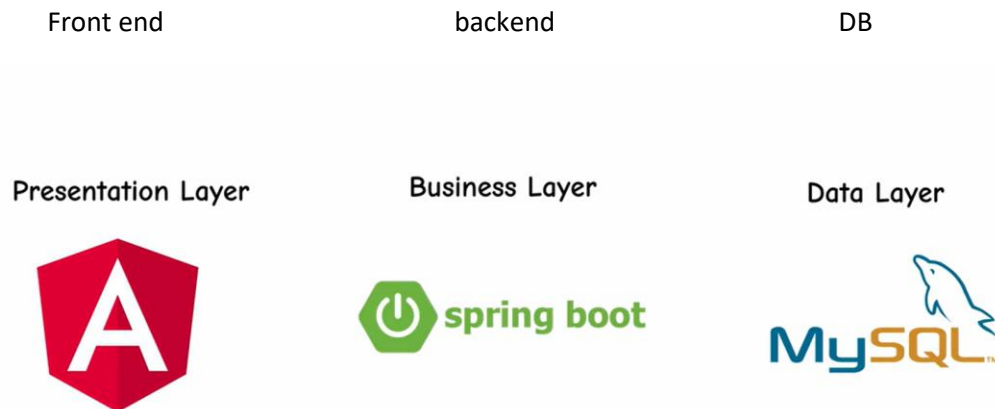


Actual 3 tier architecture :

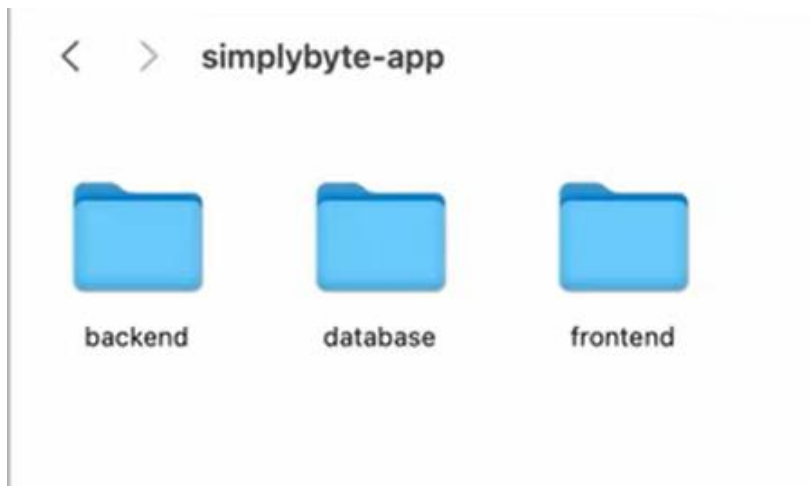


Actual flow of creation:

DB creation → Backend → Frontend

(DB pod should be created first, else if the backend pods starts it searches for DB pod to connect. If not there it crashes)

Needed yaml files :



DB : headless-service.yml, secret.yml, stateful-set.yml

Backend : deployment.yml, service.yml

Frontend : config.yml, deployment.yml, service-nodeport.yml

Prerequisites:

1. Docker

2. **Minikube** — local single-node K8s cluster
3. **kubectl** — CLI tool to talk to K8s cluster

Note : Before all, start minikube

minikube start

It spins up a single-node K8s cluster on local machine using Docker as the driver. Then `kubectl` automatically connects to this cluster.

kubectl get pods

Steps:

STEP 1 DB Creation

Starting with `stateful-set.yml` and `secret.yml` to create DB

(as DB is stateful application- that records the requests)

Note : We pass the credentials and DB_name as env variables in the `stateful-set.yml` , which the password is passed from `secret.yml` which should be written encoded shown below.

```

stateful-set.yml
13 template:
14   metadata:
15     labels:
16       app: mysql
17   spec:
18     containers:
19       - name: mysql
20         image: mysql:5.7
21         ports:
22           - containerPort: 3306
23         env:
24           - name: MYSQL_ROOT_PASSWORD
25             valueFrom:
26               secretKeyRef:
27                 name: mysql-secret
28           - name: MYSQL_DATABASE
29             value: simplybyte
30           - name: MYSQL_USER
31             value: simplybyte
32           - name: MYSQL_PASSWORD
33             valueFrom:
34               secretKeyRef:
35                 name: mysql-secret
36           - name: MYSQL_HOST
37             value: mysql
38           - name: MYSQL_PORT
39             value: 3306
40     volumeClaimTemplates:
41       - metadata:
42         name: mysql
43       spec:
44         accessModes:
45           - ReadWriteOnce
46         resources:
47           requests:
48             storage: 1Gi
49     selector:
50       matchLabels:
51         app: mysql
52     serviceName: mysql
53     updateStrategy:
54       type: RollingUpdate

secret.yml
1 apiVersion: v1
2 kind: Secret
3 metadata:
4   name: mysql-secret
5 type: Opaque
6 data:
7   mysql-root-password: c2ltcGx5Ynl0ZQ==
8
terminal
apple --zsh -- 80x24
Last login: Sat Dec 27 12:41:10 on ttys026
You have new mail.
apple@MacBook-Pro-2 ~ % echo "c2ltcGx5Ynl0ZQ==" | base64 --decode
simplybyte
apple@MacBook-Pro-2 ~ % echo "simplybyte" | base64
c2ltcGx5Ynl0ZQ==
apple@MacBook-Pro-2 ~ %

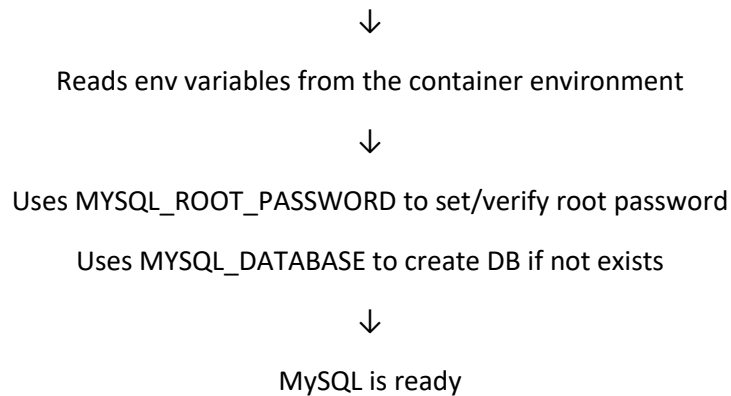
```

Question: 1. Why the `stateful-set.yml` needs DB credentials to be stored to create DB containers itself ?

DB Container starts



MySQL process boots up



!! Allow your **backend app** to connect using those same credentials every time, so credentials in StatefulSet are more important for **backend** → **MySQL connection** than for MySQL's own startup after the first time !!

Environment variable = a value stored outside your program, that the program reads when it starts (like while a container is launching it calls for the DB_creds which is stored as env variables outside the container)

ADV : No need of changing the dockerfile everytime the password is updated or changed. We easily configure the DB_creds outside without touching the code.

Need of headless-service.yml:

1. Headless Service gives each DB pod its own DNS name so other pods can reach a specific DB pod directly and the backend can directly target to the specific DB pod.
2. Normal cluster IP service randomly load balances to different pods leading to data inconsistency, it is fine if it is a stateless app.
3. StatefulSet and Headless Service are **designed to work together**.

Feature	Who provides it
Stable pod names (<code>mysql-0</code> , <code>mysql-1</code>)	StatefulSet
Stable DNS for those pod names	Headless Service

Question : 2. The backend pods read password from where to connect to DB pod ??

The Secret file is the **single source of truth** for credentials — both MySQL and Backend get their values from the same place, which ensures they always match.

1. **Secret.yml** -- start with secret file first

```
# kubectl apply -f secret.yml
```

- Before #apply secret.yml is just a file , apply stores the secret object inside cluster in **etcd**
- 1st Start reason : stateful-set searches for secret file to deploy. If no secret file → pods will fail

- To verify :

kubectl get secrets

```
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/database$ kubectl get secret
```

NAME	TYPE	DATA	AGE
mysql-secret	Opaque	1	50m

2. Headless-service.yml

#kubectl apply -f headless-service.yml

#kubectl get svc

```
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/database$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	53m
mysql	ClusterIP	None	<none>	3306/TCP	27m

(clusterIP: None) → ensures that Don't give me a virtual IP, just give me actual Pod IPs via DNS.

Note : After #minikube start → the default Kubernetes API service

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	26m

(here the **Cluster IP : none** is not set, so it got a virtual IP and not POD IP)

3. Statefulset.yml

#kubectl get sts

```
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/database$ kubectl get sts
```

NAME	READY	AGE
simplybyte-mysql	0/1	5m15s

#kubectl get pods

```
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/database$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
simplybyte-mysql-0	0/1	ContainerCreating	0	5m53s

DB POD is ready...

To access the DB pod:

```
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/database$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
simplybyte-mysql-0  1/1     Running   0           43m
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/database$
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/database$
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/database$ kubectl exec -it simplybyte-mysql-0 -- sh
sh-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.4.9 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.
```

```
mysql> use simplybyte;
Database changed
mysql> select * from sanjith;
+-----+-----+
| name | age |
+-----+-----+
| bala | 23 |
+-----+-----+
1 row in set (0.00 sec)
```

STEP 2 : Backend Creation

Deployment.yml

apiVersion: apps/v1

kind: Deployment

metadata:

name: simplybyte-backend

labels:

app: simplybyte-backend

spec:

```

replicas: 1

selector:

  matchLabels:
    app: simplybyte-backend

template:
  metadata:
    labels:
      app: simplybyte-backend

spec:
  containers:
    - name: backend
      image: simplybyte/simplybyte-backend:1.0      ----(pulls image from the public repo)
      imagePullPolicy: Always      ----(always pulls from repo)
      ports:
        - containerPort: 8090
      env:
        - name: SPRING_DATASOURCE_URL
          value: "jdbc:mysql://simplybyte-mysql-0.mysql:3306/simplybyte"

```

(Here is where the backend finds the DB to connect)

```

jdbc:mysql://  simplybyte-mysql-0 .      mysql :      3306 / simplybyte

```

```

-----

```

```

protocol      pod name      .      headless      port      database

```

```

(Java DB URL) (StatefulSet pod name)  service name

```

```

- name: SPRING_DATASOURCE_USERNAME
  value: "root"
- name: SPRING_DATASOURCE_PASSWORD

```

valueFrom:

secretKeyRef:

name: mysql-secret

key: mysql-root-password

```
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/backend$ kubectl apply -f deployment.yml
deployment.apps/simplybyte-backend created
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/backend$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
simplybyte-backend-6cdd6b5655-l5ck4	0/1	ContainerCreating	0	7s
simplybyte-mysql-0	1/1	Running	1 (117m ago)	4d19h

(see the exact difference of Pod_names, this shows the actual difference of creating pods using **stateful-set** → gives proper DNS Name, whereas **deployment** → gives random DNS Name)

Service.yml

apiVersion: v1

kind: Service

metadata:

name: simplybyte-backend

spec:

type: NodePort

selector:

app: simplybyte-backend

ports:

- port: 8090 ---> service port

targetPort: 8090 ----> container port

nodePort: 30080 -----> port to reach from outside the cluster

protocol: TCP

```

sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/backend$ kubectl apply -f service.yml
service/simplybyte-backend created
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/backend$
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/backend$
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/backend$ kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4d20h
mysql	ClusterIP	None	<none>	3306/TCP	4d20h
simplybyte-backend	NodePort	10.97.254.20	<none>	8090:30080/TCP	7s

STEP 2 : Frontend Creation

1. Starting with config.yml

The frontend config.yml stores the backend service URL as a plain text key-value pair. The frontend deployment injects this as an environment variable into the container — so the same Docker image can be deployed across dev, staging, and production just by changing the ConfigMap, without rebuilding the image. This is the K8s recommended pattern for managing non-sensitive configuration — sensitive values always go in Secrets, non-sensitive values go in ConfigMaps."

```

config.yml deployment.yml service nodeport.yml
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/frontend$ kubectl apply -f config.yml
configmap/simplybyte-frontend-config created

```

2. Deployment.yml

```

sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/frontend$ kubectl apply -f deployment.yml
deployment.apps/simplybyte-frontend created
sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/frontend$ kubectl get pods

```

NAME	READY	STATUS	RESTARTS	AGE
simplybyte-backend-6cdd6b5655-l5ck4	1/1	Running	0	120m
simplybyte-frontend-849558845f-r2nwf	0/1	ContainerCreating	0	5s
simplybyte-mysql-0	1/1	Running	1 (3h57m ago)	4d21h

```

sanjith@Sanjith-HP:~/sanjith_handson/K8s_3Tier_app/frontend$ kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4d23h
mysql	ClusterIP	None	<none>	3306/TCP	4d23h
simplybyte-backend	NodePort	10.97.254.20	<none>	8090:30080/TCP	174m
simplybyte-frontend	NodePort	10.107.98.99	<none>	80:30081/TCP	19m

Output :

localhost:30081/register

Create Account

First Name
Simply

Last Name
Byte

Email
simplybyte01@gmail.com

Password
.....

Confirm Password

Create Account

[Already have an account? Login](#)